

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 2

Listes de contacts

Ranger ses favoris, puis en faire un groupe

Famille, Amis, Bureau — filtrage et action groupée



Famille



Amis



Bureau

Périmètre Talky (Flutter) ■ Alanya-Backend

Statut Conception — aucune ligne de code écrite

Dépend de Socle de compte (migration 023, Drift v15)

Remplace docs/listes-contacts-plan.md

Date 26 juillet 2026

Avant-propos

Ce document reprend intégralement la spécification `docs/listes-contacts-plan.md`, qu'il remplace. Trois choses y ont été ajoutées ou corrigées après confrontation au code réel : la renumérotation de la migration, la version du schéma local, et l'arbitrage sur l'abstraction d'audience.

Comment lire ce document. La **partie I** décrit la fonctionnalité telle que l'utilisateur et le client la voient. La **partie II** décrit comment on la construit. On peut valider la partie I sans lire la partie II.

Table des matières

Avant-propos	1
I Approche naïve	3
1 Le besoin	4
1.1 Ce qui existe déjà	4
1.2 Ce qu'on ajoute	4
1.3 Les règles produit	4
2 Les écrans	5
2.1 Vue d'ensemble	5
2.2 Écran 1 — Contacts préférés, filtrés	5
2.3 Écran 2 — Gestion des listes	6
2.4 Écran 3 — Détail d'une liste	6
2.5 Points d'entrée	6
II Approche technique	7
3 Modèle de données	8
3.1 Deux tables	8
3.2 L'appartenance n'est pas contrainte par la base	9
4 API	10
4.1 Contrôleur — <code>src/controllers/contactListController.js</code>	10
4.2 Routes — <code>src/routes/contactLists.js</code>	10
5 Client Flutter	11
5.1 Couche données	11
5.2 Interface	12
6 Arbitrage : liste de contacts <i>n'est pas</i> audience de diffusion	13
7 Vérification	14
7.1 Backend	14
7.2 Client	14
7.3 Points ouverts	14

PARTIE I

Approche naïve

CHAPITRE 1

Le besoin

1.1 Ce qui existe déjà

Alanya a aujourd'hui une seule forme d'organisation des contacts : la liste plate des **contacts préférés**, les favoris. Elle est implémentée de bout en bout — base de données, API, cache local hors ligne, écran dédié.

Ce qui n'existe pas : aucun moyen de *regrouper* ces favoris. Un utilisateur qui a quarante contacts préférés les voit dans une seule liste alphabétique, sans distinction entre sa famille, ses amis et ses collègues.

Un mot à ne pas confondre. Dans le code actuel, « groupe » désigne uniquement les **conversations de groupe** (`conversation.isGroup`). C'est autre chose. Une liste de contacts n'est pas une conversation : c'est un rangement personnel, invisible des autres.

1.2 Ce qu'on ajoute

Permettre à l'utilisateur de ranger ses contacts préférés dans des **listes nommées** — Famille, Amis, Bureau — pour deux usages :

1. **filtrer** l'écran des contacts préférés d'un seul geste ;
2. **créer une conversation de groupe** avec toute une liste, sans resélectionner les membres un par un.

1.3 Les règles produit

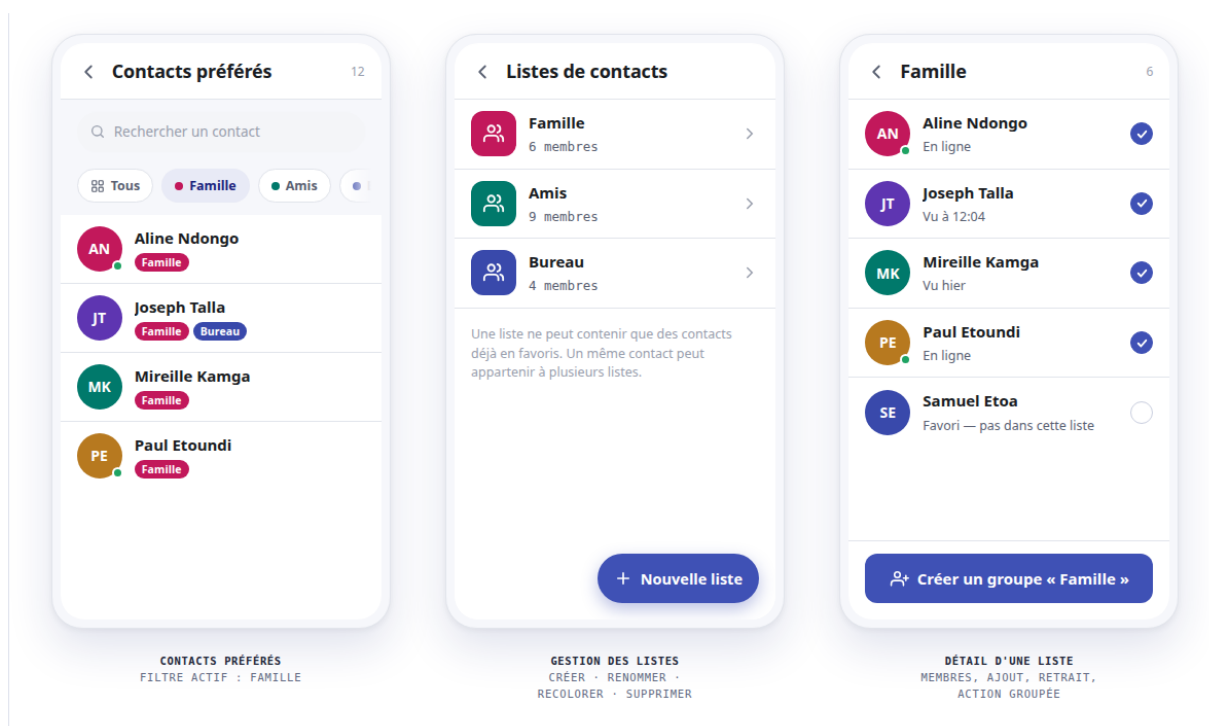
Qui peut être membre	Uniquement des contacts déjà en favoris . Une liste range ce qui est déjà rangé ; elle n'est pas un second carnet d'adresses.
Appartenance	Plusieurs à plusieurs . Un contact peut être à la fois en Famille et en Bureau.
Action groupée	Chat de groupe uniquement. Pas d'appel de groupe ni de diffusion de statut à ce stade — les tables les permettront plus tard sans modification.
Visibilité	Strictement privée . Personne ne sait dans quelle liste il est rangé, ni même qu'il l'est.
Emplacement	Puces de filtre dans l'écran des contacts préférés, <i>et</i> un écran dédié de gestion.
Navigation	Aucun sixième onglet . La barre est codée en dur à cinq onglets (<code>lib/screens/home/glass_nav_bar.dart</code>).

CHAPITRE 2

Les écrans

2.1 Vue d'ensemble

Trois écrans, un parcours : on filtre, on gère, on agit.



Maquette interactive. claude.ai/code/artifact/d15fb039

Source versionnée : docs/architecture/maquettes/listes-contacts.html

2.2 Écran 1 — Contacts préférés, filtrés

L'écran existant gagne une rangée de puces au-dessus de la liste :

- une puce « **Tous** », active par défaut ;
- une puce **par liste**, portant sa couleur ;
- une puce « **+ Gérer** » qui ouvre l'écran de gestion.

La rangée défile horizontalement — le nombre de listes n'est pas borné. Le filtre se combine avec la recherche existante : chercher « Ali » dans le filtre Famille cherche dans la famille seulement.

Chaque contact affiche les puces des listes auxquelles il appartient. C'est le seul endroit où l'appartenance multiple se voit d'un coup d'œil.

2.3 Écran 2 — Gestion des listes

Une liste par ligne : pastille de couleur, nom, nombre de membres. Créer, renommer, recolorer, supprimer. Un bouton flottant « Nouvelle liste ».

Supprimer une liste ne supprime aucun contact — seulement le rangement.

2.4 Écran 3 — Détail d’une liste

Les membres de la liste, plus les autres favoris présentés en grisé avec une case à cocher : ajouter et retirer se font au même endroit, sans écran intermédiaire.

En bas, un bouton pleine largeur : « **Créer un groupe « Famille »** ». Le nom de la liste est pré-rempli et reste modifiable. Un appui ouvre la nouvelle conversation de groupe.

Pourquoi la case à cocher plutôt qu’une feuille de sélection. Montrer les non-membres dans la même liste, en grisé, évite un aller-retour vers un écran de sélection. L’utilisateur voit d’un seul coup qui est dedans et qui pourrait y être.

2.5 Points d’entrée

1. Le profil, entrée « Listes de contacts » à côté de « Contacts préférés » (`lib/screens/profile/profile_screen.dart` méthode `_openPreferredContacts`).
2. La puce « + Gérer » de l’écran 1.

PARTIE II

Approche technique

CHAPITRE 3

Modèle de données

3.1 Deux tables

Sur le modèle exact de `preferredContact` et `conv_participants` (`migrations/001_initial_schema.sql`).

```
1  -- 024_contact_lists.sql -- listes de contacts + membres (plusieurs a
   plusieurs)
2
3  CREATE TABLE IF NOT EXISTS contact_list (
4      idList      BIGINT      NOT NULL AUTO_INCREMENT,
5      alanyaID    INT         NOT NULL,                -- propriétaire de
   la liste
6      name        VARCHAR(60) NOT NULL,
7      color       VARCHAR(9)  NULL,                  -- #RRGGBB pour la
   puce
8      created_at  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
9      PRIMARY KEY (idList),
10     UNIQUE KEY  uq_list_name (alanyaID, name),
11     KEY idx_list_owner (alanyaID),
12     CONSTRAINT fk_list_owner FOREIGN KEY (alanyaID)
13         REFERENCES users(alanyaID) ON DELETE CASCADE
14 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
15
16 CREATE TABLE IF NOT EXISTS contact_list_member (
17     idList      BIGINT      NOT NULL,
18     idFriend    INT         NOT NULL,
19     created_at  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
20     PRIMARY KEY (idList, idFriend),
21     KEY idx_clm_friend (idFriend),
22     CONSTRAINT fk_clm_list FOREIGN KEY (idList)
23         REFERENCES contact_list(idList) ON DELETE CASCADE,
24     CONSTRAINT fk_clm_friend FOREIGN KEY (idFriend)
25         REFERENCES users(alanyaID) ON DELETE CASCADE
26 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 3.1 – `migrations/024_contact_lists.sql`

Renumérotation — 023 devient 024. La spécification d’origine visait `023_contact_lists.sql`. Le numéro 023 est pris par le socle de compte, qui doit être appliqué avant. Les migrations restent appliquées à la main : il n’y a ni exécuter, ni table de suivi.

Clé primaire composite plutôt qu’un identifiant technique. `contact_list_member` n’a pas de colonne `id` : sa clé est le couple `(idList, idFriend)`. Cela interdit gratuitement le doublon d’appartenance — il n’y a rien à vérifier côté applicatif.

3.2 L'appartenance n'est pas contrainte par la base

La règle « un membre doit être un favori du propriétaire » ne peut pas s'exprimer en clé étrangère : elle porte sur le couple (`propriétaire de la liste`, `ami`), pas sur `idFriend` seul. Elle est donc vérifiée dans le contrôleur, à l'ajout, par une requête sur `preferredContact`.

Conséquence à ne pas oublier : si l'utilisateur **retire un favori**, ce contact reste dans ses listes. Deux options — le retirer en cascade applicative, ou le laisser et le filtrer à la lecture. La première est plus propre et se code en trois lignes dans `removePreferredContact` (`src/controllers/preferredContactController.js`)

CHAPITRE 4

API

4.1 Contrôleur — `src/controllers/contactListController.js`

Style copié de `preferredContactController.js` : cadrage sur `req.user.alanyaID`, `parseInt(req.params.id, 10)`, `pool.execute`, codes 400/403/404/409, et les utilitaires existants `sanitizeUrl` et `maskPresenceIfBlocked` (`src/utills/blockUtils.js`).

Handler	Rôle
<code>getLists</code>	Listes du propriétaire + <code>member_count</code> (LEFT JOIN ... GROUP BY)
<code>createList</code>	{ name, color } — 409 si <code>uq_list_name</code> est violé
<code>updateList</code>	Renommer / recolorer, avec contrôle de propriété
<code>deleteList</code>	Le CASCADE supprime les membres
<code>getListMembers</code>	JOIN vers <code>users</code> , même projection que <code>getPreferredContacts</code>
<code>addMember</code>	Vérifie la propriété et que l'ami est un favori — sinon 403
<code>removeMember</code>	—

4.2 Routes — `src/routes/contactLists.js`

Router Express avec auth sur **chaque** route et des blocs `@swagger`, comme `src/routes/contacts.js`.

GET /	<code>getLists</code>
POST /	<code>createList</code>
PUT /:idList	<code>updateList</code>
DELETE /:idList	<code>deleteList</code>
GET /:idList/members	<code>getListMembers</code>
POST /:idList/members/:friendID	<code>addMember</code>
DELETE /:idList/members/:friendID	<code>removeMember</code>

Montage dans `server.js` : `app.use('/api/contact-lists', contactListRoutes)`.

Aucun endpoint de conversation à créer. L'action « créer un groupe depuis une liste » réutilise **POST** `/api/conversations/group`, déjà en service et déjà appelé côté mobile par `createGroup` (`lib/api/chat_api.dart`, ligne 27, signature {`participantIDs`, `groupName`, `groupPhoto`}).

CHAPITRE 5

Client Flutter

5.1 Couche données

Fichier	Contenu
lib/api/contact_lists_api.dart (nouveau)	Extension ContactListsApi on TalkyApiClient, part of le client, sur le modèle de lib/api/users_api.dart. Déclarer le part dans talky_api_client.dart.
lib/talky_models.dart	Modèle ContactList : idList, name, color, memberCount, fromJson / toJson.
lib/core/db/app_database.dart	Tables LocalContactLists et LocalContactListMembers (clé composite {idList, idFriend}).
lib/core/services/local_cache_repository.dart	Section.dart listes de contacts », calquée sur watch/syncPreferredContacts.

Version du schéma local — v15 devient v16. La spécification d'origine annonçait un passage de 14 à 15. `schemaVersion` vaut aujourd'hui **14** (lib/core/db/app_database.dart, ligne 253), et le socle de compte l'amène déjà à **15**. Les listes de contacts prennent donc **16** :

```
if (from < 16) {  
  await m.createTable(localContactLists);  
  await m.createTable(localContactListMembers);  
}
```

Puis régénérer : `dart run build_runner build --delete-conflicting-outputs`. La montée doit passer par `onUpgrade`, jamais par une recreation.

Le dépôt local expose : `watchContactLists()` (flux Drift, hors ligne d'abord), `syncContactLists()` (même logique `previousIds` / `newIds` de diff-suppression que `syncPreferredContacts`), `watchListMembers(idList)` et `syncListMembers(idList)`. Les mutations passent en ligne par l'API puis re-synchronisent, comme `addContact` / `removeContact`. Les deux tables sont ajoutées à `clearSession()`.

5.2 Interface

Fichier	Objet
<code>screens/profile/preferred_contacts_screen.dart</code>	Branche de puces au-dessus de la liste; état <code>_selectedListId</code> (<code>null = Tous</code>) combiné à la recherche existante (<code>filterUsersBySearch</code>).
<code>screens/profile/contact_lists_screen.dart</code> (nouveau)	CRUD des listes.
<code>screens/profile/contact_list_detail_screen.dart</code> (nouveau)	Membres ajout / retrait, bouton « Créer un groupe ».
<code>screens/profile/profile_screen.dart</code>	Entrée « Listes de contacts ».
<code>lib/l10n/*</code>	<code>contactLists</code> , <code>createList</code> , <code>listName</code> , <code>renameList</code> , <code>deleteList</code> , <code>addToList</code> , <code>removeFromList</code> , <code>createGroupFromList</code> , <code>noLists...</code>

Composants à réutiliser, pas à réécrire. Le patron de puce existe déjà : `_buildFilterChip` dans `lib/screens/chats/chats_screen.dart`, **ligne 584** — et non 539–569 comme l'indiquait la spécification d'origine, dont la référence avait vieilli. Pour le reste : `AppAvatar`, `AppSearchField`, `EmptyState`, `AppBottomSheet`, et les jetons `AppSpacing` / `AppRadius` / `context.colors` / `context.l10n`.

CHAPITRE 6

Arbitrage : liste de contacts *n'est pas* audience de diffusion

La conception initiale (`alanya-contexte-conception.md`, §3) proposait *une seule* abstraction d'audience, réutilisée pour les segments de messages officiels, les segments de statuts officiels et les listes de contacts préférés — « ne pas la dupliquer ».

Décision prise : deux modèles distincts. Les deux objets n'ont ni le même propriétaire, ni la même échelle, ni la même nature.

	Liste de contacts	Audience de diffusion
Propriétaire	un utilisateur	un administrateur
Membres	ses favoris, énumérés	tous les comptes, décrits par un critère
Échelle	quelques dizaines	potentiellement toute la base
Nature	des lignes	une <i>requête</i>
Usage	filtrer, créer un groupe	cibler une diffusion

Les fusionner obligerait à choisir entre deux mauvaises options :

1. **matérialiser** l'audience de diffusion en lignes d'appartenance — c'est exactement le *fan-out on write* que le §9 du même document rejette ;
2. **doter** les listes personnelles d'un moteur de critères dont elles n'auront jamais l'usage.

Ce qui reste mutualisé. Deux schémas distincts, mais **un seul résolveur** côté serveur : une fonction qui prend l'un ou l'autre et renvoie la liste des destinataires. La mutualisation se fait au niveau du code d'exécution, où elle est utile, pas au niveau du schéma, où elle coûte.

CHAPITRE 7

Vérification

7.1 Backend

Appliquer `024_contact_lists.sql` sur MySQL, redémarrer, puis tester chaque endpoint via Swagger ou `curl` avec un jeton valide :

1. créer une liste → 201 ;
2. recréer le même nom → 409 ;
3. ajouter un contact **non favori** → 403 ;
4. lister les membres, en retirer un, supprimer la liste ;
5. vérifier que la suppression de la liste n'a supprimé **aucun** favori.

7.2 Client

`flutter pub get` → `dart run build_runner build --delete-conflicting-outputs` → `flutter run`.

1. Créer « Famille », y ajouter trois favoris ; la puce « Famille » filtre la liste.
2. « Créer un groupe » ouvre une conversation de groupe avec ces membres et le nom pré-rempli.
3. **Migration v15** → **v16** sur un appareil déjà installé : la mise à jour ne doit **pas** vider les données existantes.
4. **Hors ligne** : réseau coupé, listes et membres restent visibles depuis le cache Drift ; les mutations affichent un état hors ligne cohérent.
5. Retirer un favori appartenant à une liste, et vérifier le comportement retenu.

7.3 Points ouverts

1. Retrait d'un favori encore présent dans des listes : cascade applicative, ou filtrage à la lecture ?
2. Une limite au nombre de listes par utilisateur ? Rien ne l'impose techniquement, mais la rangée de puces se dégrade au-delà d'une dizaine.
3. Extensions volontairement hors périmètre, possibles sans changer les tables : appel de groupe depuis une liste, diffusion de statut restreinte à une liste.